Etapa 7: ¿quién genera la matriz?

Primera etapa posterior a la consulta docente. Algoritmos y MPI en C; análisis y gráficos en Python. Esta campaña estudia la preparación de la entrada, no mide todavía toda la aplicación.

Pregunta e hipótesis

¿Podemos evitar que el proceso 0 genere y distribuya toda la matriz? Si cada proceso calcula sus columnas directamente, eliminamos el empaquetado central y Scatterv. A cambio, se recalculan los valores simétricos y aparecen accesos por columnas; la mejora debe medirse.

Implementación: exactamente la misma matriz

El generador original visita las parejas i<j en orden. Identificamos la posición de cada pareja en esa secuencia y recuperamos el estado del generador entero sin recorrer los valores anteriores. La suma diagonal conserva el mismo orden que antes. Por ello, el resultado se puede comprobar bit a bit.

$$q(i,j) = i(2n - i - 1)/2 + (j - i - 1), \quad i < j$$

$$a_{ii} = 1 + \sum_{j \neq i} |a_{ij}|$$

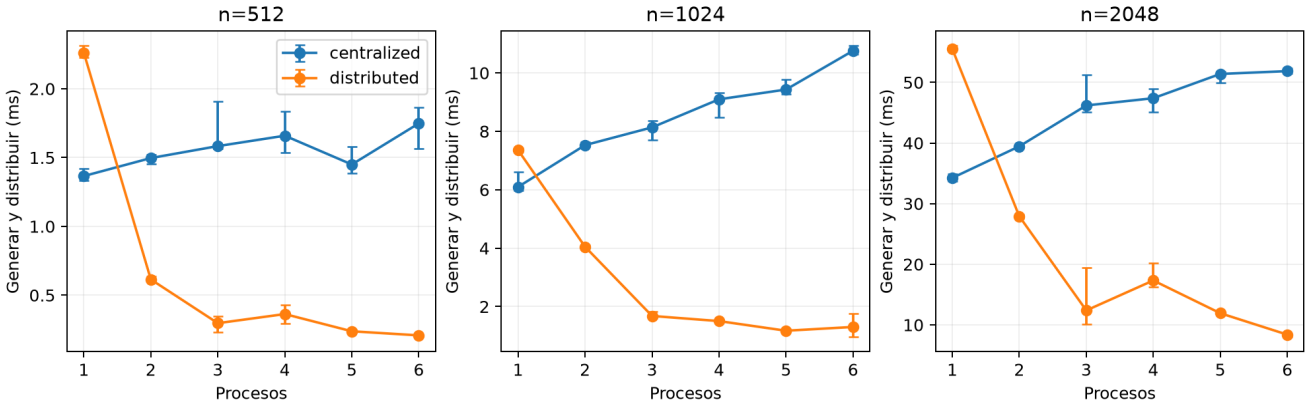
La simetría y la dominancia diagonal estricta con diagonal positiva aseguran que la matriz sea definida positiva. Cada proceso guarda sus columnas cíclicas en el formato que ya utiliza nuestro Cholesky MPI.

Protocolo

WSL, Ryzen 5 2600; 6 núcleos físicos. Tamaños 512, 1024 y 2048; P=1 a 6; semilla 2026; un calentamiento y 6 repeticiones alternadas por método. Total: 216 observaciones. Centralizado: generación en rango 0, empaquetado y Scatterv. Distribuido: generación directa local. Ambos terminan con idéntica matriz distribuida.

El cronómetro excluye reserva, arranque MPI, factorización y validación. Se toma el máximo del tiempo total entre procesos. Las fases no están sincronizadas: sus máximos incluyen esperas distintas y no deben sumarse para reconstruir el total.

Resultados y decisión provisional



Puntos: medianas. Barras: Q1-Q3; no son intervalos de confianza. Una campaña local con seis repeticiones por configuración.

n	P	Central (ms)	Distrib. (ms)	Cociente
512	1	1.366	2.262	0.60x
512	6	1.748	0.208	8.40x
1024	1	6.097	7.359	0.83x
1024	6	10.759	1.303	8.26x
2048	1	34.243	55.519	0.62x
2048	6	51.862	8.385	6.18x

Para n=2048 y P=6, el cociente entre medianas de preparación es 6.18. Este cociente compara dos métodos con el mismo P; no es el speedup de Cholesky ni el de la aplicación completa.

Comprobaciones y límites

Todas las repeticiones reprodujeron exactamente la matriz original. Se probaron también n=1,2,3,7,31 con seis procesos, incluidos procesos sin columnas. Las pruebas C y sanitizadores de las funciones nuevas pasaron. Falta repetir en sesiones independientes y medir el impacto junto con la factorización.

La reserva del banco contiene matrices adicionales para verificar igualdad; no representa la memoria mínima de una aplicación distribuida. Este experimento aún no integra el nuevo generador en el banco de tiempo completo.

¿Y si comprobar tarda demasiado?

Reconstruir todo LL transpuesta cuesta O(n³). Agregamos una comprobación complementaria con vectores de signos, reproducibles mediante una semilla separada. Para cada vector calculamos Ax y L(L transpuesta x), sin formar el producto matricial completo.

$$r_x = \frac{\|Ax - L(L^T x)\|_2}{\|Ax\|_2}$$

Con k vectores, el costo es O(kn²) y la memoria auxiliar es O(n). Esta magnitud NO es el residuo de Frobenius. Un conjunto finito de vectores puede dejar errores sin detectar: no constituye un certificado determinista ni sustituye las pruebas exhaustivas.

Las pruebas usan ocho vectores con semilla 91827, factores correctos y deliberadamente alterados, valores no finitos y elementos del triángulo superior que no pertenecen a L. También comprueban repetibilidad. El método está implementado como función independiente; los bancos históricos siguen usando el residuo completo.

Siguiente pregunta

¿La mejora al preparar la entrada reduce el tiempo que espera el usuario? El siguiente banco debe incluir reserva, generación, movimiento, factorización y salida; mantener un caso con resultado centralizado y otro distribuido; y registrar la validación por separado. Un tiempo externo adicional podrá incluir el arranque y cierre de MPI.

Ruta local pendiente

1. Integrar el flujo completo y validación escalable. 2. Matrices mayores y P=1 a 6. 3. Distribución bloque-cíclica. 4. Almacenamiento triangular. 5. Compilador y vectorización. 6. Referencia LAPACK. 7. Decidir, según evidencia y alcance defendible, algoritmo por bloques y MPI+OpenMP.

Trazabilidad y uso de IA

Fuentes: src/cholesky.c, src/compare_generation.c y tests/test_projected.c. Campaña: resultados/generacion_local_20260922, con CSV, protocolo, entorno, afinidad y copia de fuentes. Código y texto elaborados con asistencia de IA; resultados obtenidos mediante ejecución local. La defensa debe explicar las decisiones, las pruebas y sus límites.